PantryPal — Shared Pantry Tracker

Stack: React, Node.js, Express.js, MongoDB ·

1. Project Overview

PantryPal is a shared inventory application for households, hostels, offices, or teams. Members can track ingredients and supplies, update quantities, and identify low-stock or soon-to-expire items.

2. Problem Statement

People sharing a kitchen often purchase duplicate products or forget about items that are already available or close to expiry.

3. User Roles

- **Pantry Owner:** Create a pantry and manage members.
- **Pantry Member:** View inventory and update item quantities.

4. Core Features

- User registration and login.
- Create a pantry space.
- Add existing users as pantry members.
- Add inventory items.
- Record item name, category, quantity, unit, expiry date, and low-stock threshold.
- Increase or decrease quantities.
- Prevent negative quantities.
- Search and filter inventory.
- View low-stock and soon-to-expire items.
- Maintain a basic inventory activity history.

5. Suggested MongoDB Collections

- **User:** name, email, passwordHash.
- **Pantry:** name, ownerId, memberIds, createdAt.

- **PantryItem:** pantryId, name, category, quantity, unit, expiryDate, lowStockThreshold, createdBy.
- **InventoryActivity:** pantryId, itemId, userId, changeAmount, note, createdAt.

6. Suggested API Endpoints

- POST `/api/auth/register`
- POST `/api/auth/login`
- POST `/api/pantries`
- GET `/api/pantries`
- POST `/api/pantries/:id/members`
- GET `/api/pantries/:id/items`
- POST `/api/pantries/:id/items`
- PATCH `/api/items/:id`
- DELETE `/api/items/:id`
- GET `/api/pantries/:id/activity`

7. Five-Day Development Plan

- **Day 1:** Project setup, authentication, and database design.
- **Day 2:** Pantry creation and member management.
- **Day 3:** Inventory CRUD, quantity updates, search, and filters.
- **Day 4:** Low-stock alerts, expiry views, and activity history.
- **Day 5:** Testing, UI improvements, documentation, and demo.

8. Acceptance Criteria

- Pantry owners can manage members.
- Only authorized members can access pantry data.
- Quantity updates persist correctly.
- Negative quantities are prevented.
- Low-stock and expiry information is accurate.

9. Final Deliverables

Complete source code, README, MongoDB models, API documentation, test cases, screenshots, and demonstration.

Mandatory requirements

1. Create a Git repository and commit work regularly.
2. Use environment variables for configuration and keep secrets out of Git.
3. Validate inputs on both frontend and backend.
4. Implement authentication and enforce permissions on the server.
5. Use a clear and responsive UI.
6. Test all main workflows and invalid-input cases.
7. Provide a README with installation and execution steps.
8. Demonstrate the application and explain the database schema and APIs.

Final submission: Working application, frontend and backend source code, database models, `.env.example`, README, API documentation, test cases, screenshots, and a live demonstration.